



Integrating information by Kullback–Leibler constraint for text classification

Shu Yin^{1,2} · Peican Zhu² · Xinyu Wu³ · Jiajin Huang⁴ · Xianghua Li² · Zhen Wang² · Chao Gao^{2,3} 

Received: 20 October 2022 / Accepted: 11 April 2023 / Published online: 5 May 2023
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2023

Abstract

Text classification is an important assignment for various text-related downstream assignments, such as fake news detection, sentiment analysis, and question answering. In recent years, the graph-based method achieves excellent results in text classification tasks. Instead of regarding a text as a sequence structure, this method regards it as a co-occurrence set of words. The task of text classification is then accomplished by aggregating the data from nearby nodes using the graph neural network. However, existing corpus-level graph models are difficult to incorporate the local semantic information and classify new coming texts. To address these issues, we propose a Global–Local Text Classification (GLTC) model, based on the KL constraints to realize inductive learning for text classification. Firstly, a global structural feature extractor and a local semantic feature extractor are designed to capture the structural and semantic information of text comprehensively. Then, the KL divergence is introduced as a regularization term in the loss calculation process, which ensures that the global structural feature extractor can constrain the learning of the local semantic feature extractor to achieve inductive learning. The comprehensive experiments on benchmark datasets present that GLTC outperforms baseline methods in terms of accuracy.

Keywords Text classification · Graph neural network · Kullback–Leibler divergence · Constraint

1 Introduction

As a basic assignment, text classification is able to greatly improve the efficiency of text processing and text mining by dividing the text into predefined categories according to a specific classification standard [1]. It also provides essential methods for other NLP tasks such as text sentiment analysis [2], spam filtering [3, 4], and topic detection in social media [5]. Traditional text classification method is

mainly to manually design the feature extractor and then select the appropriate classifier for classification [6, 7]. Commonly used classifiers include Decision Tree (DT) [8], Support Vector Machine (SVM) [9], K-nearest Neighbor (KNN) [10], Multinomial Naive Bayes (MNB) [11, 12], and its improved models [13]. However, it is labor-consuming and time-costing to extract the features manually with the rapid growth of Internet information [14]. Therefore, it is of great significance to use machine automation to extract features and classify text.

Since Deep Learning (DL) can effectively learn feature representation, researchers begin to apply deep learning models to text classification tasks [15]. The method that uses deep learning models for text classification includes two steps: vectorization and model learning. More specifically, vectorization is mainly to convert text into a low-dimensional vector representation. Commonly used vectorization methods include Word2Vec [16], GloVe [17], and BERT [18]. Then, the model learns features from the vector representation of the text to classify text automatically. Currently, the commonly used deep learning models

✉ Chao Gao
cgao@nwpu.edu.cn

¹ The School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, Shannxi, China

² The School of Artificial Intelligence, Optics and Electronics (iOPEN), Northwestern Polytechnical University, Xi'an 710072, Shannxi, China

³ The College of Computer and Information Science, Southwest University, Chongqing 400715, China

⁴ Faculty of Information Technology, Beijing University of Technology, Beijing 100083, China

are Convolutional Neural Networks (CNN) [19], Recurrent Neural Networks (RNN) [20], and attention mechanism [21].

Despite the significance of deep learning methods in text classification, existing research identifies that methods using Graph Neural Networks (GNN) outperform traditional Machine Learning (ML) and deep learning methods for classifying text [22]. Currently, existing methods-based graph can be divided into two categories according to the input structure of texts [23]. Corpus-level methods build a heterogeneous graph for the entire text set, which can effectively extract word co-occurrence information from the graph, thereby extracting global structural information of texts and processing more complex text datasets [24]. Text-level methods build a text-level graph for each text, so these methods can effectively extract semantic information of texts and learn richer word features [25]. However, existing GNN-based models rarely consider both global co-occurrence information and local context information. In addition, existing methods applied for extracting the global graph structure information cannot classify new documents because the global graph structure will be changed when new texts appear in the training process.

To solve these problems, we present a Global–Local Text Classification model (GLTC) based on the idea of Kullback–Leibler (KL) constraint in this paper. First, we design a global structural information extractor. It models the text collection as a heterogeneous graph and then uses the multi-layer GCN model to implement information transfer to learn the low-dimensional text embedding representations with global structural information. Then, a local semantic information extractor is designed. It utilizes a bidirectional Gate Recurrent Unit (GRU) to model documents and learns low-dimensional document representation embeddings that incorporate the local contextual information. Finally, the KL constraint is introduced to realize the inductive learning. Specifically, we use the KL divergence to measure the distribution difference of the output between local and global parts. Then, such a difference is applied as the regularization term of the loss function. This innovation ensures that the local part of the model eventually learns the information obtained by the fusion of two parts. Compared with previous models, our model not only can fully incorporate the global feature of the text set and the local feature of the sentence, but also enables real-time classification for emerging texts. Some experiments are implemented on six benchmark datasets to highlight the performance of our method in terms of accuracy. The main contributions of this paper are as follows.

- We propose a text classification model that focuses on both global structural information and local semantics

information. Specifically, we use the multi-layer GCN to learn text representations with global structural features and the bidirectional GRU to learn text representations with local semantic features.

- To realize the classification of new texts, we introduce the KL constraint in training phase. Specifically, we use the KL divergence to make the global structure information extractor constrain the local semantic feature extractor, which realizes the inductive learning of model.
- We employ a series of experiments to display the utility of GLTC.

2 Related work

Firstly, we present methods for text classification using machine learning in this section. Since graph-based methods, as a class of deep learning methods, can better extract text structure information, we categorize the machine learning methods into three groups (traditional methods, deep learning methods, and graph-based methods) [14]. Then, we introduce related methods of contrastive learning, which is applied to constrain the feature information in our method.

2.1 Text classification

2.1.1 Traditional methods

The traditional machine learning method has been widely used in the field of text classification in the past ten years [26]. These methods first extract features and then select an appropriate classifier for classification. Logistic Regression (LR) is one of the earliest classification methods [27]. Currently, text classification algorithms based on improved NB are popular [28]. Wang et al. propose a Multinomial Naive Bayes (MNB) tree based on information gain measure, which reduces time consumption while improving classification accuracy [12]. MNB requires the independence assumption of dependent attributes, so Jiang et al. propose a method named SEMNB, which uses the average of all weighted single-dependency polynomial estimates to alleviate the independence assumption of MNB and improve performance [13]. However, since the potential value in the feature space needs to be evaluated by a frequentist, NB is often limited by data scarcity [29].

The core of KNN is to find the category of K points closest to the test data to predict its category. Due to the high computational complexity of traditional KNN, Zhou et al. use a clustering algorithm to process the training dataset and use the weight value to represent the

importance of each training sample. This algorithm reduces time consumption while improving classification accuracy [30]. In addition, when the distribution of data categories is unbalanced, KNN has low prediction accuracy for rare categories. Therefore, a method called NWKNN sets different weights for different categories to improve the classification performance of the imbalanced corpus [31]. However, the performance of KNN and its improved algorithms is related to the design of the distance function, causing such algorithms is dependent on the distribution of data [32].

SVM finds an optimal hyperplane that divides different categories of data in the feature space to classify the text [33]. Joachims et al. propose the Transductive Support Vector Machine (TSVM) for text classification, using a transductive approach to use the test set as an additional source of information to improve classification performance [34]. However, SVM is sensitive to missing data, so it has higher requirements for datasets [35].

Decision tree is a classification method that constructs a tree structure based on attributes to make decisions. Su et al. design an information gain-based decision tree that improves classification accuracy while reducing time complexity and being scalable to datasets with a large number of attributes [36]. Fast Decision Tree (FDT) is a fast text classification method based on DT, which uses the idea of data partition and feature pre-selection to speed up the induction of the decision tree [37]. In addition, Bigi proposes a KL-based method to realize text classification in view of word frequency matching, in which the KL-based information matching mechanism provides a way in the field of text classification [38]. However, with the rapid growth of Internet information, it is labor-consuming and time-costing for these traditional methods to classify a large number of texts [14].

2.1.2 Deep learning methods

Convolutional neural networks (CNN) are frequently employed in classifying texts due to their superb local information acquisition ability. Kalchbrenner et al. design a dynamic CNN to solve the issue of long-distance word interaction that cannot be solved by traditional CNN [39]. More specifically, they adopt a dynamic pooling approach for text classification, which effectively captures the long-term dependencies of words. Kim et al. design the TextCNN based on convolution operations to capture the essential information of sentences in different windows sizes in order to better learn the local correlation of sentences [40].

Compared with the CNN-based classification model, the recurrent neural networks (RNN)-based model has a more significant advantage in understanding long semantics

because the specified cell of RNN can identify the time series relationship of context. Liu et al. come up with the LSTM-based information-sharing models called TextRNN for text classification with multi-task learning [41]. Compared with the standard LSTM model, the Tree-LSTM model based on the tree structure proposed by Tai et al. has different forget gate in each subunit [42]. And these subunits can learn richer semantic information and text representation embeddings by selectively trading off the information.

With the widespread focus on attention mechanisms and transformers [21], pre-trained language models are used for some text classification methods [43]. The attention mechanism in these model can represent the different language segments weights in the form of a vector. Yang et al. invent a different attention mechanism for sentences and words, which has the ability to provide different attention to sentences and words with different importance in texts [44]. LEAM uses a joint embedding vector of words and labels to better learn the text representation and then further adds the attention mechanism, capturing the dependence of words in texts [45].

The above methods only take text sequences as model inputs. In fact, the sentence or text structure can be better captured if the available graph information is used as an additional input.

2.1.3 Graph-based methods

Graph neural network (GNN) can effectively classify the text by training the graph as the input data, which can be applied to node-level, edge-level, and graph-level tasks [22]. More and more researchers use GNN for text classification tasks due to its outstanding structural representation ability. Some studies build a graph structure for corpuses to obtain the global information. TextGCN model is proposed based on GCN to learn the features of words and texts by constructing a global heterogeneous graph based on word co-occurrence [24]. To obtain richer information, Liu et al. propose TensorGCN by constructing multi-graphs for corpuses based on semantics, syntax, and sequences to learn node embeddings based on the intra-graph propagation and inter-graph propagation [46]. Aiming at the sparse problem of short text, the HGAT model is designed by building heterogeneous graphs based on texts, topics, and entities to realize complex interaction of information [47]. In order to associate the global information and local semantic information efficiently, Fan et al. use a GraphSEAT containing the bidirectional GRU encoder to obtain the global information from the constructed document-entity heterogeneous graph and the local semantic information from text sequences, respectively [48].

Though these corpus-level models can effectively capture global information from text data, they cannot accomplish inductive learning on new texts. Therefore, some researchers aim to solve such problems by constructing text-level graph structures. TextLevelGNN builds a text-level graph for each text to share global parameters [25]. Then, a message-passing system is proposed for obtaining the information of contexts. The TextING model additionally uses sliding windows to capture word co-occurrence relationships in texts and considers generating a novel graph by creating edges based on such co-occurrence relationships [49]. And a gated GNN is applied to learn text embedding by aggregating the neighbor information. However, these corpus-level models do not consider the structural features of the text sets.

2.2 Contrastive learning

Contrastive learning (CL) is widely used in deep learning because it can provide a high-quality sample space for data augmentation. More specifically, CL first adopts the data augmentation techniques to construct pairs of positive samples and pairs of negative samples. Then, it improves the similarity between the embedding of positive samples and the difference between the embedding of negative samples through contrast constraints.

Currently, most methods based on the contrastive learning regard the combined similarity between the maximum positive instances and minimum negative instances as the target training model. Chen et al. propose a simple contrastive learning framework based on the visual representation called SimCLR, which constructs positive samples for images through data enhancement methods such as cropping and grayscale [50]. Then, the cosine similarity between the target and positive samples is maximized, while the cosine similarity between the target and negative samples is minimized. In contrast, SimCSE additionally introduces a multi-layer perceptron (MLP) to calculate the correlation of samples based on SimCLR [51]. Gao et al. explore the necessity of contrastive learning in unsupervised or supervised natural language processing tasks [52]. Their model uses dropout as the way to realize the data augmentation and then uses the other data outside the batch as negative examples of comparative learning. In addition, some studies achieve the contrastive learning by constructing pairs of positive samples and learning only the similarities of positive samples. For example, Fang et al. design a presentation learning method based on an autoencoder called Cert, which implements data augmentation through back-translation and fine-tuning the pre-trained language model by predicting whether two augmented sentences are from the same sentence [53]. To solve the over-fitting problems of deep neural network,

R-drop expands the output of model by using drop method and avoids the over-fitting problem by constraining the output similarity of two dropout models [54]. Chen et al. propose a dual contrastive learning framework, i.e., DualCL, for text classification [55]. DualCL treats the parameters of the classifier as label-aware-based augmented samples and performs contrastive learning by minimizing the similarity between the input samples and augmented samples.

3 Formulation of GLTC

In this section, we propose a text classification model (GLTC) that combines global and local information based on the KL constraint. The model consists of three parts as a whole, i.e., Global Structural Feature Extractor (GSFE), Local Semantic Feature Extractor (LSFE), and comparison and integration part. In the GSFE part, we construct a document-word heterogeneity graph based on the word co-occurrences from corpuses. Then, we input the graph structure into a multi-layer GCN to learn relationships between the various texts. In the LSFE part, we adopt a bidirectional GRU to process a single text sequence to learn the semantic feature of texts. In the training process of comparison and integration part, the output of GSFE is used as a positive sample to constrain the output of LSFE. So the local information model can learn the global structure information at the same time. In the prediction stage of comparison and integration part, only LSFE part is used to predict texts, which not only combines structural information with semantic information but also realizes the inductive learning of new texts prediction. Figure 1 shows the general structure of proposed model GLTC.

3.1 Global structural feature extractor

In the global structural feature extraction part, we convert a text into a heterogeneous graph and then use the GCN model to update the node representation. Based on word-text connections, we model each text as a corpus-level heterogeneous graph $G = (V, E)$, where $V = \{v_w, v_d\}$ is the nodes set in the graph, v_w and v_d represent the word nodes set and text nodes set in G , respectively, and E is the graph edges set that connect word and text nodes. When a text contains a certain word, there is an edge between the text and the word, and the weight of the edge is determined by TF-IDF [56]. TF-IDF believes that the significance of a word increases with the frequency of the word and decreases with the frequency of other words. In other words, if a word frequently appears in a certain text and less frequently appears in other texts, the word can well represent the characteristics of the text. TF-IDF is

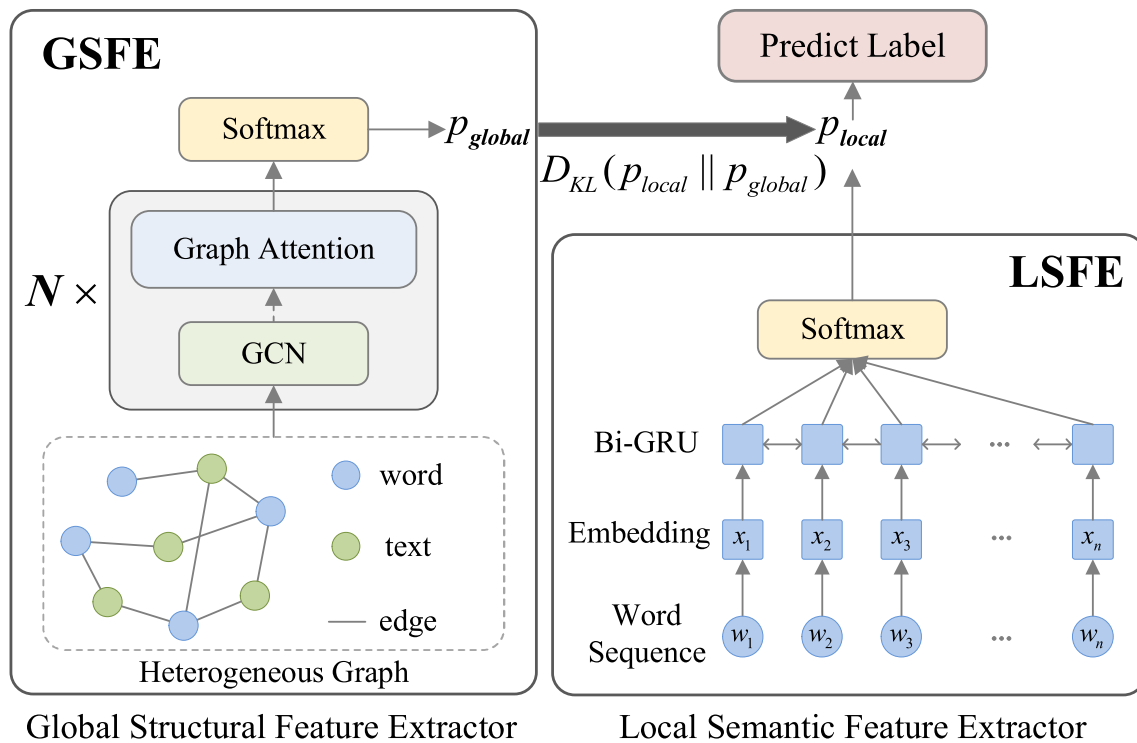


Fig. 1 Architecture of GLTC text classifier. In global structural feature extraction part, the GCN model is used to update representation of nodes in the text-word heterogeneous graph. In local semantic feature extractor part, the Bi-GRU model is used to process

text sequence data and extract local information from texts. Besides, KL divergence acts as a regularization term for the loss function to make GSFE constrain the training of LSFE, which realize inductive learning of the model

calculated in two parts, where TF stands for word frequency, IDF stands for inverse document word frequency, and TF-IDF is the product of these two parts. The calculation process is as Eq. (1) shows:

$$\begin{aligned} \text{TF}(w) &= \frac{n_w}{n} \\ \text{IDF}(w) &= \log\left(\frac{\#D}{\#D_w + 1}\right) \end{aligned} \quad (1)$$

$$\text{TF-IDF}(x) = \text{TF}(x) * \text{IDF}(x)$$

where n_w is the number of times the word w appears in a text, n is the total number of words contained in the text, $\#D$ represents the entire number of text contained in corpora, and $\#D_w$ represents the number of text containing the word w . In addition, A and D , respectively, denote the adjacency and degree matrix of the corpus-level heterogeneous graph.

Then, the heterogeneous graph G is input into the multi-layer GCN model. Since a single-layer GCN can only learn information from direct neighbor nodes, and there is no direct connection between text and text in the graph, we stack multiple GCN layers to learn high-order neighborhood information. For initial node features, the initial embedding of word nodes is represented by the Glove vector, and the initial embedding of text nodes is the

average of all word embedding vectors in the text. We denote the feature matrix of all nodes as $X \in R^{N \times d}$, where N represents the total number of nodes included in the graph, and d represents the node embedding dimension. The multi-layer GCN model takes A , D , and X as inputs, and then, the model updates the representation of all nodes. A GCN sub-layer and a graph attention layer make up each GCN layer. The information update process in the GCN sub-layer is as follows:

$$L^{(j+1)} = \sigma(\tilde{A}H^{(j)}W^{(j)}) \quad (2)$$

where σ represents the activation function, $\tilde{A} = D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$ represents the normalized symmetric adjacency matrix, $H^{(j)}$ represents the input of the j^{th} layer GCN model, $W^{(j)}$ represents the current GCN layer's weight matrix, $L^{(j+1)}$ represents the output of the j^{th} layer GCN model.

GCN learns the embedding texts representation in a graph using the weighted aggregation of the representation of the text node itself and the representation of the word nodes connected to the text node. Since different words have different importance to texts, we introduce a graph attention layer to learn appropriate neighbor node weights and use a self-attention mechanism to calculate neighbor node weights a_{vu} . The specific calculation process is:

$$e_{vu} = \sigma(W \cdot [h_v || h_u])$$

$$a_{vu} = \text{softmax}(e_{vu}) = \frac{\exp(e_{vu})}{\sum_{i \in N_v} \exp(e_{vi})} \quad (3)$$

where v represents the current node to be updated, u is one of its neighbor nodes, h_v and h_u are the embeddings of nodes v and u , W represents the attention weight matrix, and σ represents the activation function. The representation of node v is updated by aggregating its neighbor node features, finally. The update formula is:

$$h'_v = \sigma \left(\sum_{u \in N_v} a_{vu} \cdot h_u \right) \quad (4)$$

where h'_v is the output vector of the j^{th} layer GCN model, which aggregates all important information of neighbor nodes. After going through a multi-layer GCN model, the text node embedding learned by aggregating global information is denoted as $global(v)$.

3.2 Local semantic feature extractor

To learn richer contextual information, we employ a bidirectional GRU model to extract local semantic features for a single text object. GRU adopts a gating mechanism to transmit sequence state information, which is made up of two components: reset gate and update gate. The specific update process is denoted as Eq. (5).

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \phi(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \quad (5)$$

where σ and ϕ are the activation functions, x_t represents the input embedding, h_t and h_{t-1} are the hidden layer state of the model at time t and the previous moment, W_z , W_r and W_h are the weight matrices, b_z , b_r and b_h are bias.

The one-way GRU model can only process one-way sequence, which may cause the model to lose some important information. Therefore, we adopt a bidirectional GRU model, which retains important information in the context of the predicted word, and then learns the embedding representation of the text based on this information. We input the text sequence $d = \{w_1, w_2, \dots, w_n\}$ into a bidirectional GRU model, which embeds contextual information into word vectors by combining information from both directions. The forward GRU processes the text sequence from w_1 to w_n , and the resulting hidden layer state $\vec{h}_n$ is the text vector representation. The backward GRU processes the text sequence from w_n to w_1 , and the resulting hidden state $\overleftarrow{h}_1$ is the text vector representation.

Finally, the two hidden layer states are concatenated into a text representation containing local semantic features

$$\text{local}(v) = \begin{bmatrix} \vec{h}_n, \overleftarrow{h}_1 \end{bmatrix}.$$

3.3 Comparison and integration

3.3.1 Training phase

To fuse global structural information and local semantic information, we employ an attention mechanism to aggregate $global(v)$ and $local(v)$. The process is formulated as Eq. (6).

$$\begin{cases} o_g = W_g \cdot global(v) \\ o_l = W_l \cdot local(v) \\ u_g = \tanh(W_1 o_g + b_1) \\ u_l = \tanh(W_2 o_l + b_2) \\ a_g = \frac{\exp(v^T u_g)}{\exp(v^T u_g) + \exp(v^T u_l)} \\ a_l = \frac{\exp(v^T u_l)}{\exp(v^T u_g) + \exp(v^T u_l)} \\ z = \sum a_l o_l + a_g o_g \end{cases} \quad (6)$$

where o_g and o_l are the linearly transformed global information document representation and local information document representation, a_g and a_l are the global information weight and local information weight value learned by the attention mechanism, W_g , W_l , W_1 and W_2 are trainable weight matrices, b_1 and b_2 are bias terms, and z is the aggregated global structural information and local semantics information.

Then, the Softmax function is employed to calculate the predicted label and the cross-entropy loss function relies on the true labels to calculate the corresponding predicted label loss. The calculation process is:

$$P' = \text{softmax}(Wz + b)$$

$$\text{Loss}_1 = - \sum_{i \in D_{\text{train}}} y_i \cdot \log P'_i \quad (7)$$

where Loss_1 represents the original cross-entropy loss function, D_{train} is the set of texts for training, W represents the trainable weight matrix, b represents the bias term, P' represents the probability distribution of the category, and y is the true label.

The above process of fusing the representations of global and local for classification prediction can be expressed by the probability formula:

$$p(c | d) = \mathbb{E}_G \sum_{z', z''} p(c | z', z'') \cdot p(z' | d) \cdot p(z'' | G) \quad (8)$$

where $\mathbb{E}_G$ denotes the expectation of randomly sampling a heterogeneous graph G , $p(z' | d)$ denotes the representation z' of a text d randomly sampled by the LSFE, $p(z'' | G)$ denotes the text representation z'' obtained by randomly sampling heterogeneous graphs G with GSFE, $p(c | z', z'')$ denotes the label probability distribution learned by the fusion of the two sampled text representations using the attention mechanism, and $p(c | d)$ denotes the final classification label of text d .

However, the GSFE part puts all training and test set nodes into the graph during the construction phase. When making classification predictions, this part can only classify the test samples contained in the graph and cannot classify new samples. To address this issue, we consider injecting the global structural information in GSFE into LSFE during training and directly using LSFE for prediction in the final prediction stage. As shown in Fig. 2, we introduce KL constraint in the training phase, regard GSFE as a data enhancer for positive samples, and compare the text representation learned by GSFE with the text representation learned by LSFE.

KL divergence can be the quantification of the disparity between probability distributions, which is used to measure the degree of difference between the approximate distribution of the data and the original distribution of the data. The larger the difference between the two probability distributions, the larger the KL divergence value calculated from the two probability distributions. Therefore, for the

loss function, we introduce the KL divergence loss as a regularization term to jointly train the model. By minimizing the KL divergence, the document representation learned by the LSFE contains the global structural information of the GSFE. First, we transform the global document representation $\text{global}(v)$ and the local document representation $\text{local}(v)$ through a layer of nonlinear transformation to obtain two probability distribution representations p_{global} and p_{local} , the process can be expressed as:

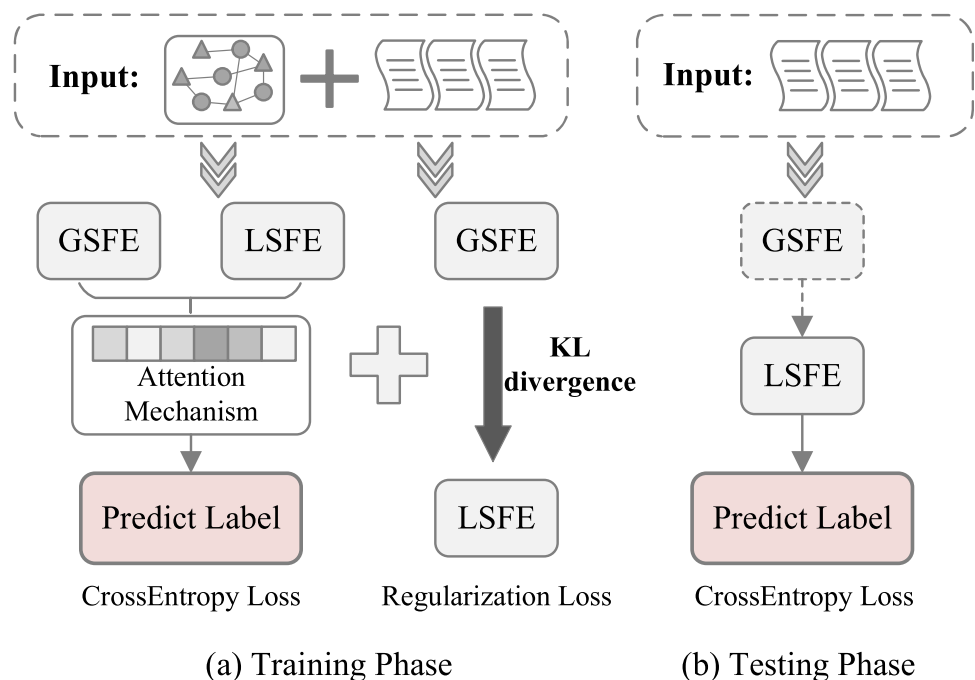
$$\begin{aligned} p_{\text{global}} &= \text{softmax}(W_{\text{global}} \cdot \text{global}(v) + b_{\text{global}}) \\ p_{\text{local}} &= \text{softmax}(W_{\text{local}} \cdot \text{local}(v) + b_{\text{local}}) \end{aligned} \quad (9)$$

Then, we use p_{global} to constrain p_{local} so that the global representation probability distribution is approximately the same as the local representation probability distribution. $D_{\text{KL}}(p_{\text{local}} \| p_{\text{global}})$ is the one-way KL divergence value computed with p_{global} constrained by p_{local} , added to the original cross-entropy loss function as the regularization term. Therefore, the final loss function is Eq. (10):

$$\text{Loss}_{\text{final}} = \text{Loss}_1 + \alpha \cdot D_{\text{KL}}(p_{\text{local}} \| p_{\text{global}}) \quad (10)$$

where $\text{Loss}_{\text{final}}$ represents the final loss, D_{KL} represents the regularization term, and α is the parameter of the regularization term. In summary, during the training process, the KL divergence is used as the regularization term of the loss function and the GSFE part is used to constrain the LSFE part. So the final LSFE part of the model also contains global structure information.

Fig. 2 Differences between the training and prediction phases of the model. We use both GSFE and LSFE and use GSFE as a data augmentor for LSFE in training phases, but only use LSFE for prediction in prediction stage



3.3.2 Testing phase

In the above training phase, we use the text representation learned by GSFE to constrain the text representation learned by LSFE. The distribution of text representation learned by GSFE is approximately equal to the text representation learned by LSFE. Therefore, the training process after adding KL constraints can be expressed in Eq. (11):

$$\begin{cases} p(c | d) = \mathbb{E}_G \sum_{z', z''} p(c | z', z'') p(z' | d) p(z'' | G) \\ p(z' | d) = p(z'' | G) \quad \text{KL constraints} \end{cases} \quad (11)$$

After introducing the KL constraint, the text representation $p(z' | d)$ learned based on LSFE is equal to the text representation $p(z'' | G)$ learned based on GSFE. Therefore, in the testing phase, Eq. (11) is combined, and the following process can be obtained as:

$$\begin{aligned} p(c | d) &\stackrel{\text{step 1}}{=} \mathbb{E}_G \sum_{z', z''} p(c | z', z'') p(z' | d) p(z'' | d) \\ &\stackrel{\text{step 2}}{\approx} \sum_{z', z''} p(c | z', z'') p(z' | d) p(z'' | d) \\ &\stackrel{\text{step 3}}{\approx} \sum_z p(c | z) p(z | d) \end{aligned} \quad (12)$$

The following explains this derivation step by step:

- Step 1 is obtained by combining Eq. (11).
- Step 2 is because the learned text representation after merging is no longer affected by the GSFE. That is, it is no longer affected by the heterogeneous graph, so $\mathbb{E}_G$ is eliminated.
- Step 3 is to sample two text representation samples from the trained LSFE randomly. Without considering the impact of dropout, we can consider $p(z' | d) = p(z'' | d)$. The final text expression z is composed of z' and z'' .

Therefore, in the testing phase, the text representation learned by LSFE can be directly used for classification.

4 Experiments

In order to confirm the effectiveness of the GLTC model, we conduct a number of experiments and analyses on six public datasets in this part.

4.1 Experimental settings

4.1.1 Evaluation datasets

To confirm the effectiveness of GLTC, this paper conducts experiments on six public datasets, including sentiment datasets, movie review datasets, medical datasets, and news datasets.

- **SST1** and **SST2**¹ [57] are from the Stanford Sentiment Treebank. According to the degree of positivity and negativity, SST1 is divided into 5 fine-grained categories, and SST2 is divided into 2 categories.
- **MR**² [58] is a movie review dataset that contains both positive and negative sentiment categories.
- **Ohsumed**³ is from the MEDLINE medical database, which classifies medical abstracts and titles into 23 cardiovascular disease categories.
- **R8** and **R52**⁴ are subsets of the news dataset included in Reuters News, with 8 categories and 52 categories.

We normalize cases and eliminate the slice of tokens to preprocess each benchmark dataset. For Ohsumed and R8, we eliminated special characters, stop words, and low-frequency words (i.e., words that appear no more than 5 times). For the other datasets, we only eliminated special characters due to the shorter text length in these datasets. The information of each dataset after preprocessing is shown in Table 1, where the corresponding elements, i.e., Texts, Train, Test, Words, Classes, and $\langle L \rangle$ denote the number of texts, the number of texts in train sets, the number of texts in test sets, the number of words appearing in the dataset corpus, the number of categories, and the overall average sentence length in each dataset, respectively.

4.1.2 Baseline models

This paper conducts experiments on a variety of baseline text classification methods, including traditional deep learning text classification methods, pre-trained language model methods, and graph neural network methods:

Traditional machine learning methods

- **TF-IDF + LR** uses TF-IDF as the feature of the text and uses LR as the classifier.
- **TF-IDF + SVM** uses TF-IDF as the feature of the text and finds an optimal hyperplane that divides several data types in the feature space.

¹ <https://nlp.stanford.edu/sentiment/code.html>.

² <http://www.cs.cornell.edu/people/pabo/movie-review-data/>.

³ <http://disi.unitn.it/moschitti/corpora.htm>.

⁴ <https://www.cs.umb.edu/smimarog/textmining/datasets/>.

Table 1 Information of each dataset

Dataset	Texts	Train	Test	Words	Classes	$\langle L \rangle$
SST1	11855	8544	1101	4683	5	19.17
SST2	9613	7792	1821	4516	2	19.62
MR	10662	7108	3554	18764	2	19.44
Ohsumed	7400	3357	4043	14157	23	121.59
R8	7674	5485	2189	7688	8	65.72
R52	9100	6536	2568	8892	52	69.82

Traditional deep learning methods

- **TextCNN** [40] is based on CNN to classify texts, which can extract key information and local dependencies of sentences. The authors propose several different TextCNN models, and we select the CNN-no-static to compare with our proposed method.
- **TextRNN** [41] is based on RNN to classify texts. RNN are good at processing sequential data and capturing long-range sentence dependencies.
- **FastText** [59] is an efficient text vector library for fast text classification tasks created by the Facebook Research.

Pre-trained language model methods

- **Transformer** [21] is composed of attention mechanisms and mainly consists of two parts: encoder and decoder. We choose the encoder part and then add an output layer to implement the text classification task.
- **BERT** [18] is a novel pre-training method based on the encoder part of the transformer with a bidirectional structure. There are a series of BERT models released by Google, and we choose the base BERT and add an output layer as a comparison model.

Graph neural network methods

- **TextGCN** [24] is a model based on GCN for classifying texts. It constructs a heterogeneous graph of the corpus from word co-occurrence relationships in each text and then uses the GCN with two hidden layers to learn node representations by message passing.
- **TextLevelGNN** [25] creates a graph and updates node representations using a graph neural network for each text.
- **TextING** [49] does not take into account the initial weights of nodes. Firstly, a graph is contrasted for each text in corpora, and then, the model introduces a message-passing mechanism for information interaction between nodes to learn node and text representations.

- **TW-TGNN** [60] proposes a two-window graph construction method-based graph neural network model with initial weights to capture word correlations.
- **TEGNN** [61] is a method that uses topological augmentation to exploit the structural features of corpora and sentence graphs for text classification.

4.1.3 Parameter settings

After a series of preliminary experiments, we choose Glove⁵ to initialize word embeddings, and the word dimension is 300. For GLTC, Adam (Adaptive momentum) is chosen as the optimizer, with the learning rate set at 0.005 and the dropout set at 0.5. The regularization factor of the loss function is set to 0.5, and the hidden layer dimension is set to 96. During training, we select 10% of the training set at random to serve for validation. In addition, the training process is terminated when the loss of validation set has not declined during 10 consecutive iterations or as soon as 64 iterations have finished. We conduct each experiment 10 times and present the average results. For the baseline model, we keep the parameter settings consistent with the default settings in the original article or implementation [49].

4.2 Results and analysis

We assess the effectiveness of GLTC by comparing it with (1) SOTA text classification models on six public datasets and (2) its variants on three well-performing datasets. Then, we analyze the decreasing trend of the loss function value during training. Next, we analyze the effect of different numbers of GCN layers on GLTC classification accuracy. Finally, we analyze the dataset distribution to propose scenarios for using the GLTC model.

4.2.1 Model's performances

We compare the GLTC with related methods in text classification research on accuracy metrics. In addition, we perform a corrected paired two-tailed t test on the results with a significance level of $p = .05$. Table 2 displays the results of the experiment, where the column W/ T/ L (Win/ Tie/ Loss) indicates that compared with the baseline, GLTC wins/ ties/ loses on W/T /L datasets. Additionally, we perform the Wilcoxon signed-ranks test on each pair of algorithms based on the results in Table 2. Due to space issues, we show the detailed comparison results of the eight best-performing algorithms in Table 3, where the significance level above the diagonal is $\alpha = 0.05$, and the

⁵ <http://nlp.stanford.edu/data/glove.6B.zip>.

Table 2 The Accuracy on Test Set of Six Public Datasets. We underline the suboptimal results and bold the best results for each dataset. The symbol • means that our results significantly improve compared with the baseline. The symbol ◦ means that our results significantly deteriorate to the baseline

	SST1	SST2	MR	Ohsumed	R8	R52	Average	W/T/L
TF-IDF + LR	0.4158 •	0.8285 •	0.7457 •	0.5467 •	0.9376 •	0.8695 •	0.7240	6/0/0
TF-IDF + SVM	0.4256 •	0.8203 •	0.7559 •	0.5318 •	0.9402 •	0.8782 •	0.7264	6/0/0
TextCNN	0.4231 •	0.8683 •	0.7772 •	0.5844 •	0.9517 •	0.8756 •	0.7240	6/0/0
TextRNN	0.4236 •	0.8060 •	0.7768 •	0.4927 •	0.9631 •	0.9052 •	0.7279	6/0/0
FastText	0.3608 •	0.8423 •	0.7507 •	0.5801 •	0.9622 •	0.9279 •	0.7378	6/0/0
Transformer	0.3509 •	0.8512 •	0.7509 •	0.6087 •	0.9644 •	0.9015 •	0.7379	6/0/0
BERT	0.4016 •	0.8569 •	0.7613 •	0.6046 •	0.9678 •	0.9135 •	0.7510	6/0/0
TextGCN	0.4036 •	0.8599 •	0.7683 •	0.6847 •	0.9705 •	0.9355 •	0.7740	6/0/0
TextlevelGNN	0.4607 •	0.8762 •	0.7613 •	0.6940 •	0.9780 •	0.9461 •	0.7861	6/0/0
TextING	0.4673 •	<u>0.8894</u>	0.7979 •	0.7044 •	0.9806	0.9556	0.7992	3/3/0
TW-TGNN	<u>0.4701</u>	0.8893	0.7975 •	0.7045 •	0.9679 •	0.9521 •	0.7969	4/2/0
TEGNN	0.4666 •	0.8798 •	<u>0.7988</u> •	<u>0.7089</u> •	<u>0.9814</u>	0.9583 ◦	0.7989	4/1/1
GLTC	0.4750	0.8898	0.8029	0.7182	0.9817	<u>0.9577</u>	0.8042	—/—/—

Table 3 Results of the Wilcoxon signed-ranks test. The symbol ◦ indicates that the method in the corresponding column has improved relative to the method in the corresponding row, and the symbol •

indicates that the method in the corresponding row has improved relative to the method in the corresponding column

Method	FastText	Transformer	BERT	TextGCN	TextlevelGNN	TextING	TW-TGNN	TEGNN	GLTC
FastText	—			◦	◦	◦	◦	◦	◦
Transformer		—	◦	◦	◦	◦	◦	◦	◦
BERT			—	◦	◦	◦	◦	◦	◦
TextGCN	•	•	•	—	◦	◦	◦	◦	◦
TextlevelGNN	•	•	•	•	—	◦		◦	◦
TextING	•	•	•	•	•	—			◦
TW-TGNN	•	•	•	•			—		◦
TEGNN	•	•	•	•				—	◦
GLTC	•	•	•	•	•	•	•		—

significance level below the diagonal is $\alpha = 0.03$. We can see that the text classification methods based on the graph model significantly outperform the traditional deep learning methods and the pre-trained language models on all datasets. It is shown that graph model-based approaches can better perform text categorization by capturing more complex textual information. Overall, the accuracy of GLTC outperforms the baseline methods on five datasets.

Specifically, GLTC significantly enhances classification performance on all datasets in comparison with conventional deep learning methods and pre-trained language models (TextCNN, TextRNN, FastText, Transformer, and BERT). The main reason is that these approaches are all focused on analyzing a single text and do not address the connection between texts. For graph-based methods, GLTC significantly improves the classification accuracy on all datasets compared to TextGCN and TensorGCN. The main

reason is that although these two models can learn text representations with global structural information, they do not address the interaction of words within texts. Compared with the baseline models TextING, GLTC improves classification accuracy on most datasets. The main reason is that TextING focuses only on the local semantic information and not the global information. Compared to the best baseline method TEGNN, GLTC improves classification accuracy on most datasets, which is 1.80% higher on the SST1 dataset, 1.14% on the SST2 dataset, 0.51% on the MR dataset, and 1.31% on the Ohsumed dataset. The results show that the GLTC can better integrate global structural features and local semantic features to extract richer text representation information, which improves text classification performance.

Table 4 Results of Ablation Experiments

	MR	Ohsumed	R8
GLTC	0.8029	0.7182	0.9817
GLTC_G	0.7768	0.4927	0.9631
GLTC_L	0.7674	0.6836	0.9707

Bold represent the best results for each dataset

4.2.2 Ablation experiments

We perform ablation experiments in this part to investigate the significance of global and local modules. We remove the global feature extractor as GLTC_G and local feature extractor as GLTC_L, respectively, and conduct experiments on three well-performing datasets (MR, Ohsumed, and R8). The classification accuracy of the test set on three datasets is shown in Table 4.

Compared with GLTC_G, GLTC demonstrates a considerable improvement in classification accuracy on three

datasets, which is 2.61% higher on the MR dataset, 22.55% higher on the Ohsumed dataset, and 1.86% higher on the R8 dataset. GLTC_G only relies on the local sequence information to model the text, so it cannot learn the text representation that incorporates global structural information. Especially, the improvement of GLTC is very significant on the Ohsumed dataset, which further demonstrates the importance of global structural information in the text classification of specialized fields. Compared with GLTC_L, the classification accuracy of GLTC has a significant improvement on the three datasets, which is 3.52%, 3.46% and 1.1% higher on the MR, Ohsumed and R8 datasets, respectively. GLTC_L only relies on the heterogeneous graph constructed by word co-occurrence to model text features, so it cannot learn fine-grained text features. More importantly, GLTC_L cannot classify new texts since the heterogeneous graph structure changes when new documents appear.

Fig. 3 Model training loss descent process. The GLTC model typically converges within 50 iterations

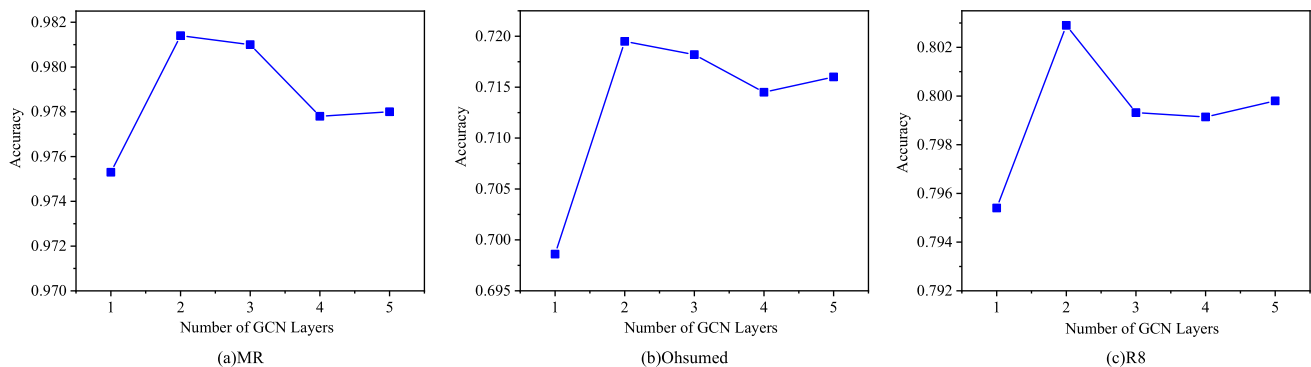
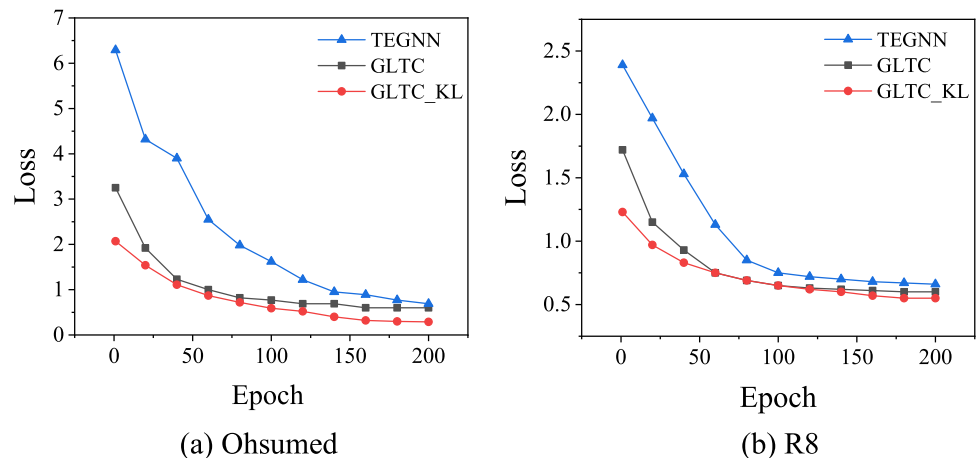
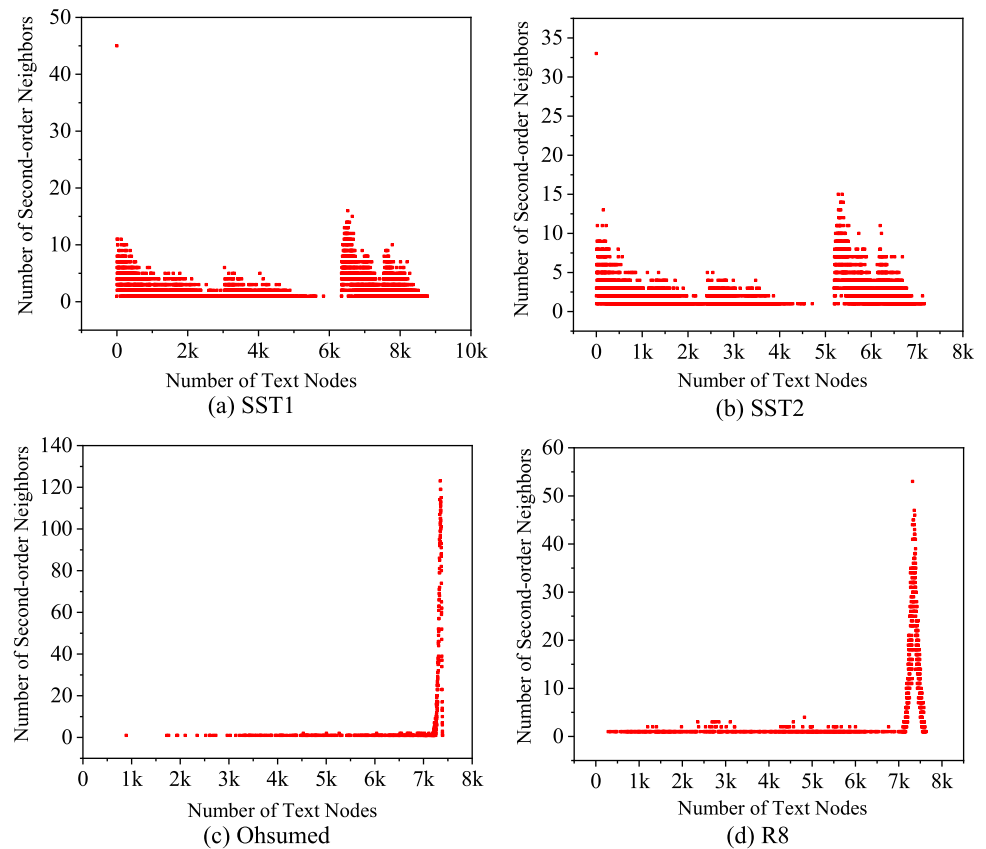


Fig. 4 Accuracy of GLTC employing various GCN layer counts on the three datasets, respectively. The best performance of our model is achieved with two GCN layers

Fig. 5 Distribution characteristics of the four types of data presented. The distribution of the number of indirectly connected texts in SST1 and SST2 datasets is more even, and there is a more concentrated distribution of the number of indirectly connected texts in Ohsumed and R8 datasets



4.2.3 Loss function analysis

This section analyzes the drop in loss value during the model training. We compare with two methods (TEGCN and GLTC_KL), where GLTC_KL is the GLTC model without KL divergence, and we experiment on two datasets with better performance. The experiment results of the experiment are shown in Fig. 3.

It can be seen that on the Ohsumed and R8 datasets, the training speed of the TextING is slower, and the TEGCN model needs more iterations to converge. Compared with GLTC_KL, GLTC adds KL divergence as a regular term to the loss function. Therefore, in the early stage of training, the loss value of GLTC model is larger. As the quantity of iterations grows, the GLTC model converges faster than the GLTC_KL model, usually within 50 iterations. The above results show that the model that integrates global structural information and local semantic information is faster to train. In addition, including a KL regularization term in the loss function not only allows the model to predict new texts but also accelerates the model's convergence rate.

4.2.4 Effects of the number of GCN layers

This section studies the effect of the number of GCN layers on the classification accuracy of the model. We conduct experiments on GLTC with various numbers of GCN layers using three effective datasets (MR, Ohsumed, and R8). According to the experiment results shown in Fig. 4, the classification accuracy is significantly impacted by the number of GCN layers. The model performs best on these datasets when there are two GCN layers. The reason is that when the model uses a two-layer GCN, the nodes in the graph can obtain second-order neighborhood information. Specifically, text nodes can aggregate not only the information of word nodes directly connected but also the information of text nodes indirectly connected. When there is only one GCN layer, GLTC has poor classification performance on these datasets. The explanation for this is when the model uses a single-layer GCN, only word nodes that are directly linked to text nodes can access information from them, and it is difficult to obtain high-order neighborhood text node information. Furthermore, as the number of GCN layers increases in model, the performance of the model does not improve significantly while consuming a lot of memory. Therefore, choosing an appropriate number

of GCN layers can efficiently learn text representations and improve classification accuracy.

4.2.5 Dataset analysis

In this part, we select four typical datasets SST1, SST2, Ohsumed, and R8 for statistical analysis. Among them, SST1 and SST2 belong to the social media field data, which have the characteristics of short text and sparse data. Ohsumed and R8 are datasets in the medical field and the news field, respectively, with long text and features belonging to special fields. The statistical outcomes are displayed in Fig. 5.

The results of the experiment display that SST1 and SST2 have similar data distribution characteristics. The number of documents indirectly connected by words in these datasets is evenly distributed and has a large span. It shows that after social media text is converted into a graph structure, the connection between text nodes is relatively sparse, and there are independent document nodes. Therefore, graph training with GCN models is difficult to provide additional useful information for these isolated nodes. At the same time, due to the sparsity of the graph structure, the information that those indirectly connected text nodes can obtain is also limited. The distributions of the Ohsumed and R8 datasets are quite different from those of SST1 and SST2. We can see from Fig. 5 that the amount of indirectly connected text is relatively concentrated. Most texts can be indirectly connected with other texts, and there are few isolated document nodes in Ohsumed and R8. The minimum number of documents indirectly connected is also close to 1000 in Ohsumed, indicating that after the text in this special field is converted into a graph structure, the connection between text nodes is tight, and there is no isolated text node. Therefore, GCN models can learn the global structural information well for such corpora with closely related texts to learn more accurate text representations, which improves classification performance.

5 Conclusion

We propose a KL constraint-based GLTC model for text classification by KL divergence that fuses global structural features and local semantic features in this paper. Firstly, a global structural feature extractor is designed to transform text sets into a heterogeneous graph model, and multi-layer GCN network models are utilized to learn document embedding representations. Then, a local semantic feature extractor is designed to model a single text sequence with a bidirectional gated recurrent unit model. Finally, an improved loss function strategy is designed based on KL constraints for model training. The final text representation

learned by the local semantic feature extractor contains both global structural information and local semantic information. And in the prediction stage, inductive learning is achieved by using a local semantic feature extractor to make classification predictions for new texts. Comprehensive experimental on multiple benchmark datasets demonstrates that the GLTC model can reach better performance. However, the GLTC model is sensitive to the parameters of the regularization term. Hence, our future work will concentrate on the model's robustness to explore advanced methods which can better fuse the global structural information and local semantic information.

Acknowledgements This research was supported by the Key Program for International Science and Technology Cooperation Projects of China (No. 2022YFE0112300), the National Natural Science Foundation of China (Nos. 61976181, 62271411, 62073263), Natural Science Basic Research Plan in Shaanxi Province of China (No. 2022JM-325), the Technological Innovation Team of Shaanxi Province (No. 2020TD-013), the Natural Science Foundation of Hebei Province (Grant No. U22B2036), the Fundamental Research Funds for the Central Universities (D5000230112), the Tencent Foundation and XPLOER PRIZE.

Data Availability The datasets generated during and/or analyzed during the current study are available in the Experiments section.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Lai S, Xu L, Liu K, Zhao J (2015) Recurrent convolutional neural networks for text classification. In: Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, pp 2267–2273
2. Whitelaw C, Garg N, Argamon S (2005) Using appraisal groups for sentiment analysis. In: Proceedings of the 14th ACM International Conference on Information and Knowledge Management, pp 625–631
3. Yu D, Chen CLP, Xu H (2022) Fuzzy swarm control based on sliding-mode strategy with self-organized omnidirectional mobile robots system. *IEEE Trans Syst, Man, Cybern: Syst* 52(4):2262–2274
4. Barushka A, Hajek P (2020) Spam detection on social networks using cost-sensitive feature selection and ensemble-based regularized deep neural networks. *Neural Comput Appl* 32(9):4239–4257
5. Zhu J, Wang C, Gao C, Zhang F, Wang Z, Li X (2022) Community detection in graph: an embedding method. *IEEE Trans Netw Sci Eng* 9(2):689–702
6. Chen L, Jiang L, Li C (2021) Using modified term frequency to improve term weighting for text classification. *Eng Appl Artif Intell* 101:104215
7. Jiang L, Li C, Wang S, Zhang L (2016) Deep feature weighting for naive Bayes and its application to text classification. *Eng Appl Artif Intell* 52:26–39

8. Quinlan JR (1996) Learning decision tree classifiers. *ACM Comput Surv* 28(1):71–72
9. Forman G (2008) BNS feature scaling: an improved representation over tf-idf for svm text classification. In: *Proceedings of the 17th ACM Conference on Information and Knowledge Management*, pp 263–270
10. Tan S (2006) An effective refinement strategy for KNN text classifier. *Expert Syst Appl* 30(2):290–298
11. Zhang L, Jiang L, Li C (2016) A new feature selection approach to naive Bayes text classifiers. *Int J Pattern Recognit Artif Intell* 30(02):1650003
12. Wang S, Jiang L, Li C (2015) Adapting naive Bayes tree for text classification. *Knowl Inf Syst* 44:77–89
13. Jiang L, Wang S, Li C, Zhang L (2016) Structure extended multinomial naive Bayes. *Inf Sci* 329:346–356
14. Minaee S, Kalchbrenner N, Cambria E, Nikzad N, Chenaghlu M, Gao J (2021) Deep learning-based text classification: a comprehensive review. *ACM Comput Surv* 54(3):1–40
15. Lu G, Gan J, Yin J, Luo Z, Li B, Zhao X (2020) Multi-task learning using a hybrid representation for text classification. *Neural Comput Appl* 32(11):6467–6480
16. Mikolov T, Sutskever I, Chen K, Corrado G, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: *Proceedings of the 26th International Conference on Neural Information Processing Systems* 2:3111–3119
17. Pennington J, Socher R, Manning CD (2014) Glove: Global vectors for word representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp 1532–1543
18. Devlin J, Chang M, Lee K, Toutanova K (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Lang Technol* 1:4171–4186
19. Yamashita R, Nishio M, Do RKG, Togashi K (2018) Convolutional neural networks: an overview and application in radiology. *Insights Imaging* 9(4):611–629
20. Cho K, van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, Bengio Y (2014) Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp 1724–1734
21. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp 6000–6010
22. Malekzadeh M, Hajibabae P, Heidari M, Zad S, Uzuner O, Jones JH (2021) Review of graph neural network in text classification. In: *Proceedings of the 2021 IEEE 12th Annual Ubiquitous Computing, Electronics and Mobile Communication Conference*, pp 0084–0091
23. Li X, Wu X, Luo Z, Du Z, Wang Z, Gao C (2022) Integration of global and local information for text classification. *Neural Comput Appl* 35:1–16
24. Yao L, Mao C, Luo Y (2019) Graph convolutional networks for text classification. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33:7370–7377
25. Huang L, Ma D, Li S, Zhang X, Wang H (2019) Text level graph neural network for text classification. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp 3444–3450
26. Jiang L, Zhang L, Li C, Wu J (2018) A correlation-based feature weighting filter for naive Bayes. *IEEE Trans Knowl Data Eng* 31(2):201–213
27. LaValley MP (2008) Logistic regression. *Circulation* 117(18):2395–2399
28. Jiang L, Zhang L, Yu L, Wang D (2019) Class-specific attribute weighted naive Bayes. *Pattern Recogn* 88:321–330
29. Kowsari K, Jafari Meimandi K, Heidarysafa M, Mendu S, Barnes L, Brown D (2019) Text classification algorithms: a survey. *Information* 10(4):150
30. Yong Z, Youwen L, Shixiong X (2009) An improved KNN text classification algorithm based on clustering. *J Comput* 4(3):230–237
31. Tan S (2005) Neighbor-weighted k-nearest neighbor for unbalanced text corpus. *Expert Syst Appl* 28(4):667–671
32. Sahgal D, Parida M (2014) Object recognition using gabor wavelet features with various classification techniques. In: *Proceedings of the Third International Conference on Soft Computing for Problem Solving: SocProS 2013(1)*:793–804
33. Joachims T (2005) Text categorization with support vector machines: Learning with many relevant features. In: *Machine Learning: ECML-98: 10th European Conference on Machine Learning Chemnitz, Germany, April 21–23, 1998 Proceedings*, pp 137–142
34. Joachims T et al (1999) Transductive inference for text classification using support vector machines. In: *Proceedings of the Sixteenth International Conference on Machine Learning* 99:200–209
35. Li Q, Peng H, Li J, Xia C, Yang R, Sun L, Yu PS, He L (2022) A survey on text classification: from traditional to deep learning. *ACM Trans Intell Syst Technol* 13(2):41
36. Su J, Zhang H (2006) A fast decision tree learning algorithm. *AAAI Conf Artif Intell* 6:500–505
37. Vateekul P, Kubat M (2009) Fast induction of multiple decision trees in text categorization from large scale, imbalanced, and multi-label data. In: *2009 IEEE International Conference on Data Mining Workshops*, pp 320–325
38. Bigi B (2003) Using Kullback–Leibler distance for text categorization. In: *Advances in Information Retrieval: 25th European Conference on IR Research*, pp 305–319
39. Kalchbrenner N, Grefenstette E, Blunsom P (2014) A convolutional neural network for modelling sentences. In: *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics* 1:655–665
40. Kim Y (2014) Convolutional neural networks for sentence classification. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp 1746–1751
41. Liu P, Qiu X, Huang X (2016) Recurrent neural network for text classification with multi-task learning. In: *Proceedings of the 25th International Joint Conference on Artificial Intelligence*, pp 2873–2879
42. Tai KS, Socher R, Manning CD (2015) Improved semantic representations from tree-structured long short-term memory networks. In: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing* 1:1556–1566
43. Chang W-C, Yu H-F, Zhong K, Yang Y, Dhillon IS (2020) Taming pretrained transformers for extreme multi-label text classification. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp 3163–3171
44. Yang Z, Yang D, Dyer C, He X, Smola A, Hovy E (2016) Hierarchical attention networks for document classification. In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp 1480–1489
45. Wang G, Li C, Wang W, Zhang Y, Shen D, Zhang X, Henao R, Carin L (2018) Joint embedding of words and labels for text

- classification. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics 1:2321–2331
46. Liu X, You X, Zhang X, Wu J, Lv P (2020) Tensor graph convolutional networks for text classification. In: Proceedings of the AAAI Conference on Artificial Intelligence 34:8409–8416
 47. Linmei H, Yang T, Shi C, Ji H, Li X (2019) Heterogeneous graph attention networks for semi-supervised short text classification. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp 4821–4830
 48. Fan M, Cheng D, Yang F, Luo S, Luo Y, Qian W, Zhou A (2020) Fusing global domain information and local semantic information to classify financial documents. In: Proceedings of the 29th ACM International Conference on Information and Knowledge Management, pp 2413–2420
 49. Zhang Y, Yu X, Cui Z, Wu S, Wen Z, Wang L (2020) Every document owns its structure: inductive text classification via graph neural networks. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp 334–339
 50. Chen T, Kornblith S, Norouzi M, Hinton G (2020) A simple framework for contrastive learning of visual representations. In: International Conference on Machine Learning, pp 1597–1607 (2020)
 51. Chen X, Fan H, Girshick R, He K (2020) Improved baselines with momentum contrastive learning. arXiv preprint [arXiv:2003.04297](https://arxiv.org/abs/2003.04297)
 52. Gao T, Yao X, Chen D (2021) SimCSE: Simple contrastive learning of sentence embeddings. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp 6894–6910
 53. Fang H, Wang S, Zhou M, Ding J, Xie P (2020) Cert: Contrastive self-supervised learning for language understanding. arXiv preprint [arXiv:2005.12766](https://arxiv.org/abs/2005.12766)
 54. Liang X, Wu L, Li J, Wang Y, Meng Q, Qin T, Chen W, Zhang M, Liu T (2021) R-drop: regularized dropout for neural networks. *Adv Neural Inf Process Syst* 34:10890–10905
 55. Chen Q, Zhang R, Zheng Y, Mao Y (2022) Dual contrastive learning: Text classification via label-aware data augmentation. arXiv preprint [arXiv:2201.08702](https://arxiv.org/abs/2201.08702)
 56. Aizawa A (2003) An information-theoretic perspective of tf-idf measures. *Inf Process Manag* 39(1):45–65
 57. Socher R, Perelygin A, Wu J, Chuang J, Manning CD, Ng AY, Potts C (2013) Recursive deep models for semantic compositionality over a sentiment treebank. In: Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, pp 1631–1642
 58. Pang B, Lee L (2005) Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In: Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics, pp 115–124
 59. Joulin A, Grave E, Bojanowski P, Mikolov T (2017) Bag of tricks for efficient text classification. In: Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics 2:427–431
 60. Wu X, Luo Z, Du Z, Wang J, Gao C, Li X (2021) Tw-tgcn: Two windows graph-based model for text classification. In: 2021 International Joint Conference on Neural Networks, pp 1–8
 61. Song R, Giunchiglia F, Zhao K, Tian M, Xu H (2022) Graph topology enhancement for text classification. *Appl Intell* 52:1–14

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.